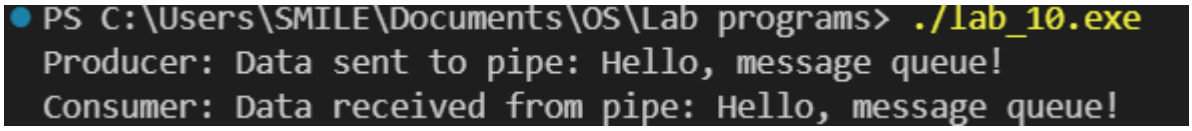
Lab program-10:

10. Illustrate the concept of inter-process communication using message queue with a c program

lab_10.c > main()

```c
#include <stdio.h>
#include <windows.h>
#include <string.h>

int main()
{
    HANDLE hReadPipe, hWritePipe;
    SECURITY_ATTRIBUTES sa;
    DWORD bytesWritten, bytesRead;

    char writeBuffer[] = "Hello, message queue!";
    char readBuffer[100];

    // Security attributes
    sa.nLength = sizeof(SECURITY_ATTRIBUTES);
    sa.lpSecurityDescriptor = NULL;
    sa.bInheritHandle = TRUE;

    // Create an anonymous pipe
    if (!CreatePipe(&hReadPipe, &hWritePipe, &sa, 0))
    {
        printf("CreatePipe failed. Error: %lu\n", GetLastError());
        return 1;
    }

    // Producer: Write to the pipe
    if (!WriteFile(hWritePipe,
                   writeBuffer,
                   strlen(writeBuffer) + 1,
                   &bytesWritten,
                   NULL))
    {
        printf("WriteFile failed. Error: %lu\n", GetLastError());
        CloseHandle(hReadPipe);
        CloseHandle(hWritePipe);
        return 1;
    }

    printf("Producer: Data sent to pipe: %s\n", writeBuffer);

    // Consumer: Read from the pipe
    if (!ReadFile(hReadPipe,
                  readBuffer,
                  sizeof(readBuffer),
                  &bytesRead,
                  NULL))
    {
        printf("ReadFile failed. Error: %lu\n", GetLastError());
        CloseHandle(hReadPipe);
        CloseHandle(hWritePipe);
        return 1;
    }

    printf("Consumer: Data received from pipe: %s\n", readBuffer);

    // Cleanup
    CloseHandle(hReadPipe);
    CloseHandle(hWritePipe);

    return 0;
}
```

OUTPUT:


PS C:\Users\SMILE\Documents\OS\Lab programs> ./lab_10.exe
Producer: Data sent to pipe: Hello, message queue!
Consumer: Data received from pipe: Hello, message queue!